Advanced Simulation of Rotary Kite Turbine Systems in Julia

1. Executive Synthesis

Simulating Rotary Airborne Wind Energy Systems (RAWES) necessitates modeling the coupled aero-structural dynamics of the airborne rotor and the Tensile Rotary Power Transmission (TRPT) network. Unlike Ground-Generation (pumping) systems, Kite Turbines continuously transmit mechanical power. The physical limits of this transmission are governed by the inter-ring torsional deformation of the TRPT structure and the distributed inertial mass of the system. This document provides a rigorous mathematical framework and Julia implementation for simulating these dynamics, tracking the critical segmented collapse limits, generating 3D physical visualizations of the tensegrity network, and exporting the time-domain results for analysis.

2. Chain-of-Verification (CoV) Presentation

- **Phase 1: Initial Synthesis:** The system is driven by aerodynamic torque generated by the crosswind motion of the rigid blades/kites. This torque accelerates the rotor mass and is transmitted down the tether to the ground station.
- **Phase 2: Verification (Physical Contradictions in Legacy Models):** Early models assumed arbitrary constant inertia and a continuous elastic shaft. **This is physically incorrect.** 1. *Inertial Reality:* The inertia (I_{total}) must be derived from the physical mass of the blades (m_{blade}) at the rotor radius (R), plus the distributed mass of the TRPT polygon spacer rings (m_{ring}).
  2. *Torsional Collapse:* The TRPT is a tensegrity structure. Torque capacity is achieved by separating the lines using n_{rings}. The critical failure mode ($\delta_{crit} = 180^\circ$) does not apply to the total twist of the shaft (α_{tot}), but strictly to the localized twist angle between two consecutive polygon rings (α_{seg}). If $\alpha_{seg} \to 180^\circ$, the lines cross, the tensegrity collapses inward, and torque transmission instantly falls to zero.
- **Phase 3: Final Integrated Output:** We construct a differential-algebraic equation (DAE) system in Julia using DifferentialEquations.jl. We calculate true physical inertia, discretize the TRPT into segments defined by the polygon rings, continuously audit α_{seg} against δ_{crit}, and output a 3D geometric rendering of the airborne network state to visually audit structural integrity.

3. Engineering Rigor, Terminology & Metrics

- P : Net mechanical power transmitted to the ground (W).
- ρ : Air density ($1.225\ kg/m^3$).
- v_w : Free-stream wind velocity at the hub altitude (m/s).
- d_t, L_0 : Tether line diameter (m) and total unstretched length (m).
- n_{lines} : Number of distinct tether lines in the TRPT.
- n_{rings}, m_{ring} : Number of polygon spacer rings and mass per ring (kg).
- n_{blades}, m_{blade} : Number of lifting blades/kites and mass per blade (kg).
- R : Radius of the airborne ring/hub (m).
- α_{tot} : Total twist angle of the entire TRPT structure (rad).
- α_{seg} : Twist angle between two consecutive TRPT polygon rings ($\alpha_{tot}/(n_{rings}+1)$).
- δ_{crit} : The $180°$ (π radians) critical torsional limit *per segment*.
- I_{total} : Total physically derived rotational inertia ($kg \cdot m^2$).

4. Mathematical Formulations

A. Physically Derived Inertia

The inertia is not assumed; it is a direct function of the system's physical mass distribution.

Assuming the blades and rings operate at the nominal radius R:

$$I_{blades} = n_{blades} \cdot m_{blade} \cdot R^2$$

$$I_{rings} = n_{rings} \cdot m_{ring} \cdot R^2$$

$$I_{total} = I_{blades} + I_{rings} + I_{pto}$$

(Note: I_{pto} represents the ground generator/Power Take-Off inertia).

B. Segmented TRPT Torque Transmission

The torque (τ) is governed by the stiffness of a single segment between rings. A segment has length $L_{seg} = L_0/(n_{rings} + 1)$. Utilizing a linearized elastic approximation for the tensegrity structure:

$$k_{seg} = \frac{E_{modulus} \cdot (\pi \cdot (d_t/2)^2) \cdot n_{lines} \cdot R^2}{L_{seg}}$$

$$\tau_{transmitted} = k_{seg} \cdot \sin(\alpha_{seg})$$

Constraint: If $\alpha_{seg} \geq \pi$, structural collapse occurs ($\tau_{transmitted} \to 0$).

5. Software Project: TRPT Julia Simulator

To ensure scientific reproducibility, the simulation is structured as a self-contained Julia software project. This separates the environment configuration from the core physics engine.

5.1. Repository Structure

```
TRPT_Simulator/
├── README.md
├── Project.toml          # Auto-generated by Julia Pkg
├── Manifest.toml         # Auto-generated by Julia Pkg
├── output/               # Directory for generated CSVs and Plots
│   ├── .gitkeep
└── src/
    └── TRPT_Kite_Turbine_Sim.jl  # Main simulation script
```

5.2. README.md & Instructions

TRPT Kite Turbine Simulator

A high-fidelity, rigid-body dynamic simulator for Rotary Airborne Wind Energy Systems utilizing Tensile Rotary Power Transmission (TRPT). This tool calculates dynamic state responses, physical inertia, and tracks the critical segmented collapse limit (δ_{crit}) of the tensegrity network.

Installation Instructions

1. **Install Julia:** Download and install Julia v1.10+ from julialang.org.
2. **Setup Project Environment:**

Open your terminal, navigate to the TRPT_Simulator folder, and launch the Julia REPL:
cd TRPT_Simulator
julia

3. **Instantiate Dependencies:**
   Inside the Julia REPL, press] to enter the Package Manager, then activate and instantiate the environment:
   (@v1.10) pkg> activate .
   (TRPT_Simulator) pkg> add DifferentialEquations DataFrames CSV Plots
   (TRPT_Simulator) pkg> instantiate

   Press Backspace to exit the package manager.

Run Instructions

To execute the simulation and generate the data arrays and 3D visual plots, run the main script from your terminal:

```
julia --project=. src/TRPT_Kite_Turbine_Sim.jl
```

Outputs will be automatically saved to your local working directory.

5.3. Source Code (src/TRPT_Kite_Turbine_Sim.jl)

```
# src/TRPT_Kite_Turbine_Sim.jl

using DifferentialEquations
using DataFrames
using CSV
using Plots

# 1. Define Strict Physical Parameters
struct TRPTParams
    rho::Float64        # Air density (kg/m^3)
    v_w::Float64        # Wind speed (m/s)
    R::Float64          # Rotor/Ring radius (m)
    n_lines::Int64      # Number of TRPT lines
    d_tether::Float64   # Tether diameter (m)
    E_modulus::Float64  # Tether Young's Modulus (Pa) - e.g., Dyneema ~100 GPa
    L0::Float64         # Unstretched total TRPT length (m)
    n_rings::Int64      # Number of polygon spacer rings
    m_ring::Float64     # Mass per polygon ring (kg)
    n_blades::Int64     # Number of lifting blades/kites
    m_blade::Float64    # Mass per blade (kg)
```

```
    I_pto::Float64      # Ground generator inertia (kg*m^2)
end

# 2. Define the System Dynamics (ODE)
function trpt_dynamics!(du, u, p, t)
    # State variables:
    # u[1] = total global twist angle (alpha_tot) in radians
    # u[2] = airborne rotor rotational velocity (omega) in rad/s
    alpha_tot = u[1]
    omega = u[2]

    # TRPT Spatial Discretization
    n_segments = p.n_rings + 1
    alpha_seg = alpha_tot / n_segments    # Twist strictly between consecutive rings
    L0_seg = p.L0 / n_segments            # Length of a single segment

    # Physical Inertia Calculation
    I_blades = p.n_blades * p.m_blade * p.R^2
    I_rings  = p.n_rings * p.m_ring * p.R^2
    I_total  = I_blades + I_rings + p.I_pto

    # Aerodynamic Torque Generation (Simplified proxy for Blade Element Momentum)
    # Replaces arbitrary torque with an assumed rotor power coefficient (Cp ~ 0.15)
    tau_aero = 0.5 * p.rho * p.v_w^2 * (pi * p.R^2) * 0.15 * p.R

    # Tether Drag & Centrifugal Penalty (The 1/4 rule approximation)
    v_apparent = sqrt(p.v_w^2 + (omega * p.R)^2)
    drag_tether = 0.25 * 1.0 * p.d_tether * p.L0 * p.rho * v_apparent^2
    tau_drag = drag_tether * (p.R * 0.5) # Drag acting at average radius

    # Segmented TRPT Structural Transmission
    # ENFORCING PHYSICAL LIMIT: delta_crit is pi (180 deg) per segment
    if abs(alpha_seg) >= pi * 0.95
        # Tensegrity structure collapses; lines touch/cross.
        tau_transmitted = 0.0
    else
        # Segment stiffness
        k_seg = (p.E_modulus * (pi * (p.d_tether/2)^2) * p.n_lines * p.R^2) / L0_seg
        tau_transmitted = k_seg * sin(alpha_seg)
    end

    # Ground Station PTO (Power Take-Off) dynamics
    # Simplified as a linear damper extracting power: tau = c * omega_ground
```

```
    c_pto = 5000.0 # Damping coefficient
    omega_ground = tau_transmitted / c_pto

    # Equations of Motion
    du[1] = omega - omega_ground                          # Rate of twist accumulation
    du[2] = (tau_aero - tau_drag - tau_transmitted) / I_total      # Rotor angular acceleration
end

# 3. Initialization & Execution
println("Initializing TRPT Simulation parameters...")
# Parameters: 10m/s wind, 5m radius, 6 lines, 4mm Dyneema, 150m length
# 10 rings @ 2.5kg each, 3 blades @ 15.0kg each, 50kg*m^2 ground inertia
params = TRPTParams(1.225, 10.0, 5.0, 6, 0.004, 100e9, 150.0, 10, 2.5, 3, 15.0, 50.0)

u0 = [0.0, 1.0] # Starts with 0 twist, rotating at 1 rad/s
tspan = (0.0, 120.0) # Simulate for 120 seconds

println("Solving Differential Equations...")
prob = ODEProblem(trpt_dynamics!, u0, tspan, params)
sol = solve(prob, Tsit5(), reltol=1e-6, abstol=1e-6)

# 4. Data Extraction & Export
println("Processing simulation output data...")
n_segments = params.n_rings + 1
df = DataFrame(
    Time_s = sol.t,
    Total_Twist_deg = rad2deg.(sol[1, :]),
    Segment_Twist_deg = rad2deg.(sol[1, :] ./ n_segments),
    Rotor_Speed_rads = sol[2, :]
)

# Derive Power Output (P = tau * omega_ground)
function calc_power(alpha_tot)
    alpha_seg = alpha_tot / n_segments
    L0_seg = params.L0 / n_segments
    if abs(alpha_seg) >= pi * 0.95
        return 0.0
    else
        k_seg = (params.E_modulus * (pi * (params.d_tether/2)^2) * params.n_lines * params.R^2) /
L0_seg
        tau = k_seg * sin(alpha_seg)
        omega_g = tau / 5000.0 # Match c_pto from solver
        return (tau * omega_g) / 1000.0 # Return in kW
```

```
    end
end

df.Power_kW = calc_power.(sol[1, :])

CSV.write("TRPT_Simulation_Results.csv", df)
println("SUCCESS: Tabular data exported to 'TRPT_Simulation_Results.csv'")

# 5. Graphical Rendering (2D Time Series)
println("Generating 2D performance plots...")
p1 = plot(df.Time_s, df.Segment_Twist_deg, label="Segment Twist (α_seg)", lw=2, color=:blue,
        title="Inter-Ring Torsional Deformation", ylabel="Angle (Degrees)", legend=:topleft)
hline!(p1, [180.0], label="δ_crit (Collapse Limit)", color=:red, ls=:dash, lw=2)

p2 = plot(df.Time_s, df.Power_kW, label="Ground Power Output", lw=2, color=:green,
        title="System Power Generation", xlabel="Time (s)", ylabel="Power (kW)",
legend=:bottomright)

fig2d = plot(p1, p2, layout=(2,1), size=(800, 600))
savefig(fig2d, "TRPT_Simulation_Graphs_2D.png")
println("SUCCESS: 2D Performance graphs exported to 'TRPT_Simulation_Graphs_2D.png'")

# 6. 3D Geometric Representation of TRPT State
println("Rendering 3D tensegrity structural state...")
function render_trpt_3d(p::TRPTParams, alpha_tot::Float64)
    n_nodes = p.n_rings + 2 # Includes ground (0) and rotor (N+1)
    L_seg = p.L0 / (p.n_rings + 1)
    alpha_seg = alpha_tot / (p.n_rings + 1)

    p3d = plot(camera=(30, 45), legend=false, title="3D TRPT Physical State",
            xlabel="X (m)", ylabel="Y (m)", zlabel="Z (m)", aspect_ratio=:equal)

    X = zeros(n_nodes, p.n_lines)
    Y = zeros(n_nodes, p.n_lines)
    Z = zeros(n_nodes, p.n_lines)

    # Calculate geometric node positions based on twist
    for i in 1:n_nodes
        node_idx = i - 1
        z_pos = node_idx * L_seg
        theta = node_idx * alpha_seg # Local twist relative to ground
        for j in 1:p.n_lines
            phi = theta + (j - 1) * (2 * pi / p.n_lines)
```

```
        X[i, j] = p.R * cos(phi)
        Y[i, j] = p.R * sin(phi)
        Z[i, j] = z_pos
    end

    # Plot Polygon Spacer Rings (in black)
    if node_idx > 0 && node_idx < n_nodes - 1
        ring_x = [X[i, :]; X[i, 1]]
        ring_y = [Y[i, :]; Y[i, 1]]
        ring_z = [Z[i, :]; Z[i, 1]]
        plot!(p3d, ring_x, ring_y, ring_z, color=:black, lw=1.5)
    end
  end

  # Plot Load-Bearing Tethers (in blue)
  for j in 1:p.n_lines
    plot!(p3d, X[:, j], Y[:, j], Z[:, j], color=:blue, lw=1.0)
  end

  # Highlight Rotor/Blades (Top Ring) in Red
  plot!(p3d, [X[end, :]; X[end, 1]], [Y[end, :]; Y[end, 1]], [Z[end, :]; Z[end, 1]], color=:red, lw=3)

  return p3d
end

# Generate and save the 3D plot using the final simulated twist state
final_twist = sol[1, end]
fig3d = render_trpt_3d(params, final_twist)
savefig(fig3d, "TRPT_3D_Structure.png")
println("SUCCESS: 3D Structure exported to 'TRPT_3D_Structure.png'")
println("Simulation run complete.")
```

6. Critical Audit & Unanswered Technical Gaps

While this framework correctly captures the multi-segment geometric limits, visualizes the tensegrity constraints, and utilizes the aggregate inertial physics outlined in recent primary literature (e.g. Trevisi, Tulloch, Wacker), an engineer must recognize its bounds:

1. **Rigid Body Assumption:** This model lumps the inertia at radius R and treats the blades as rigid entities. In reality, large S1223/ram-air Sled blades exhibit significant aero-structural deformation (bending and torsion) under high centrifugal loads, which

alters the local angle of attack and degrades C_L.

2. **Simplified Aerodynamics:** The code utilizes an assumed bulk power coefficient (C_P). A true high-fidelity model requires replacing tau_aero with a Blade Element Momentum (BEM) loop that calculates induction factors iteratively across the swept annulus, accounting for the massive blockage effect of the TRPT structure itself.
3. **Lift Kite Decoupling:** This model assumes the TRPT axis (β) is maintained statically. In the real world, the inertial mass of the rings (m_{ring}) sags the tether under gravity, requiring a lift kite. The unsteady dynamics of the lift kite reacting to wind shear are not coupled here.